

Sprint 4 — Full Stack Integration, Simplified (RabbitMQ, Redis, Serilog & Kibana)

Duration: Week 8 – Week 9 (2 weeks)

Curriculum: Background Services + Event-Driven Basics + RabbitMQ (Wk 8) | Redis Caching + Serilog + Elasticsearch/Kibana + Testing (Wk 9)

App Review 4: Full Stack Project — Integrated Application (End of Week 9)

PS-02 Deliverable: A working application with one RabbitMQ messaging flow, one Redis caching case, Serilog logging visible in a Kibana dashboard, and a small passing test suite — everything installed as a standalone local service on each trainee's machine

Learning Objectives

- Understand what a background/hosted service is in ASP.NET Core — build one trivial BackgroundService before touching any messaging library.
- Understand Event-Driven Architecture at its simplest — one publisher, one consumer, one message type, no retries or dead-letter handling.
- Install RabbitMQ as a standalone local Windows service and integrate it via MassTransit — publish one domain event, consume it with `IConsumer<T>`.
- Understand Redis caching (cache-aside pattern) through a single case — doctor availability — with a TTL and one manual invalidation trigger.
- Configure Serilog with console and rolling-file sinks only, logging a small, fixed set of events.
- Ship Serilog output to a standalone Elasticsearch instance and view it in Kibana — both installed locally with no Docker or containers.
- Write a small set of xUnit + Moq unit tests covering the booking flow only.
- Write one guided integration test using WebApplicationFactory, built together as a class exercise.

What Trainees Will Build

This sprint is intentionally scoped to one working case per technology rather than full production coverage, since RabbitMQ, Redis, Serilog, and Kibana are all new to this batch. Everything is installed and run as a standalone local application on each trainee's machine. Each capability is added without changing the existing Angular or Blazor UX.

Background Service Primer (Warm-Up): Before RabbitMQ is introduced, trainees build one trivial BackgroundService (e.g. a heartbeat that logs every 10 seconds via `ILogger`, registered with `AddHostedService`) to see how code runs outside the request/response cycle. This removes the mystery before MassTransit's own hosted consumer is introduced.

RabbitMQ Setup (Standalone Install): RabbitMQ requires Erlang as a prerequisite. Both are installed directly on each trainee's machine using their official Windows installers (Erlang/OTP, then the RabbitMQ Windows installer) — RabbitMQ runs as a local Windows service, no container involved. The management UI is enabled with `rabbitmq-plugins enable rabbitmq_management` and viewed at `localhost:15672`, so trainees can watch messages arrive and clear in real time.

AppointmentBooked Event — Publisher: One message contract only: `AppointmentBookedEvent { AppointmentId, PatientName, DoctorId, ScheduledDate, TimeSlot }`. After `POST /api/appointments` saves successfully, the API calls `IBus.Publish()` — no second event type, no transaction-timing edge cases to teach.

AppointmentBooked Event — Consumer: `AppointmentBookedConsumer` implements `IConsumer<AppointmentBookedEvent>`. `MassTransit` hosts it automatically — no manual background-service wiring required. On receipt, it creates one Notification record for the Doctor. Idempotency and duplicate-event handling are dropped from this sprint as a stretch task, not a graded requirement.

Redis — Doctor Availability Cache (Only): `GET /api/doctors/{id}/availability` checks Redis key `doctors:{id}:availability:{date}` first. On a miss, data is read from SQL Server and cached with a 5-minute TTL; on a hit, it is served directly from Redis. One invalidation trigger: updating a doctor's schedule deletes that one key. The appointment slot-lock feature is removed from this sprint — it depends on a working mental model of race conditions that hasn't been built yet, and can be introduced later once Redis basics are comfortable.

Serilog Logging — Console + File: One console sink and one rolling-file sink only. Three things are logged: every API request (method, path, status), every appointment booked, and every exception. No environment-based log-level switching and no long list of structured properties per event.

Log Monitoring — Elasticsearch + Kibana (Standalone Install): Kibana needs Elasticsearch behind it to store and index logs. Both are installed as standalone local services from their official Windows .zip archives — no Docker, no container. Elasticsearch is started first (`bin\elasticsearch.bat`, or installed as a Windows service via `bin\elasticsearch-service.bat` install); on first run it generates a password for the elastic user and an enrollment token for Kibana, printed to the terminal. Kibana is then started (`bin\kibana.bat`) and connected using that enrollment token. Serilog ships logs to Elasticsearch using the Elasticsearch sink (`Serilog.Sinks.Elasticsearch`), and trainees view, search, and filter them in the Kibana dashboard at `localhost:5601`. This is the heaviest piece of infrastructure in the sprint, so trainees should expect the extra setup time and the additional memory Elasticsearch needs on their machines.

Unit Tests — Booking Flow Only: `AppointmentServiceTests` covering 2–3 scenarios tied to the booking flow built this sprint (e.g. valid booking succeeds, past-date booking throws, notification is created). Every repository dependency is mocked with Moq. Coverage of every service class and the full 5-tests-per-class target are dropped for this sprint.

Integration Test — Guided Lab: One `WebApplicationFactory<Program>` test class is built together as a class exercise, with the trainer providing the skeleton and

trainees filling in the assertions for a single scenario: POST /api/appointments with valid data returns 201.

App Review 4 — Assessment Focus

- A trivial BackgroundService runs and logs on its own schedule, showing the trainee understands hosted services before MassTransit is introduced.
- RabbitMQ messaging works end-to-end — AppointmentBooked event is published on booking, consumed, and a notification is created.
- Doctor availability is served from Redis on the second request, confirmed via logs, with cache invalidation working on schedule update.
- Serilog logs are structured and visible in the Kibana dashboard, with Elasticsearch running as the underlying log store.
- The booking-flow unit tests pass, covering at least one happy path and one exception scenario.
- The guided integration test passes against a test database.
- Trainee can explain, in their own words, why the consumer runs in the background and what problem the message queue solves.